Now we need to tell Compose to use these files instead of explicitly setting the variables directly. We do this using the env_file directive:

```
version: '3'

services:

  web:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - .:/usr/src/app
    env_file:
      - .env/development/database
      - .env/development/web

  redis:
    image: redis

  database:
    image: postgres
    env_file:
      - .env/development/database
```

We could have named the environment files anything we liked, but I chose a simple naming scheme that makes sense. Similarly, you are free to use whatever file structure and naming conventions you like for the environment variable files (currently under .env), so long as you refer to them correctly in the Compose file.

With this small refactor done, we're ready to create our development and test databases using the standard Rails command bin/rails db:create, targeting the command at our web service:

```
$ docker-compose run --rm web bin/rails db:create
```

In this case, we used run --rm rather than exec because the currently running web container won't have the newly added environment variables set until we restart. The new, throwaway container that runs this command will.

You should see the following output showing our databases have been created successfully:

```
Database 'myapp_development' already exists
Created database 'myapp_test'
```

You'll notice that the myapp_development database already exists, which is a bit strange since this is the first time we've said to create it. That's because the postgres image will automatically create a default database when it's first started; if set, it uses the value of the POSTGRES_DB environment variable as